



Manual do Programador - Gestão de Atividades

Guia técnico da arquitetura atual, questionários mensais, Supabase, permissões, APIs, testes, Git e publicação da área de Atividades.

Ramo	Gestão de Atividades
Destinatários	Programadores e responsáveis técnicos
Repositório	https://github.com/MenteMovimento/central-mente-movimento
Site	https://central-mente-movimento.vercel.app
Atualização	29/07/2026

1. Visão geral da Central

A Central MenteMovimento junta Sócios, Utentes, Cibersegurança e Atividades num único projeto publicado na Vercel e ligado a um único projeto Supabase. A autenticação é centralizada; os dados continuam separados por tabelas e contexto funcional.

- Repositório oficial: <https://github.com/MenteMovimento/central-mente-movimento>
- Site de produção: <https://central-mente-movimento.vercel.app>
- Branch principal: main.
- Build de produção: `npm run build`.
- Diretório publicado pela Vercel: `public`.
- Não publicar ficheiros `.env`, bases SQLite locais, anexos reais, exports com dados pessoais ou chaves privadas.

2. Variáveis e segurança

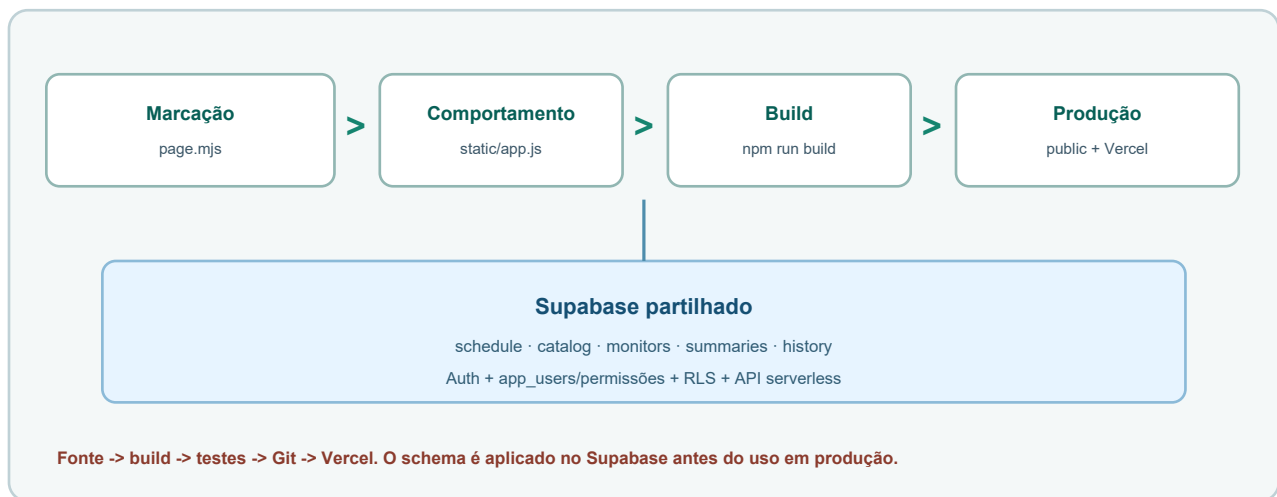
- `SUPABASE_URL` e `SUPABASE_ANON_KEY` podem ser usadas pelo frontend.
- `SUPABASE_SERVICE_ROLE_KEY` é secreta e deve ficar apenas em Environment Variables da Vercel ou ambiente local protegido.
- `SUPABASE_SECRET_KEY` é usada pelo backend Python de Utentes quando aplicável.
- Depois de mudar variáveis na Vercel, fazer redeploy.
- Se uma chave secreta for exposta, gerar uma nova no Supabase, atualizar Vercel e revogar a antiga.

3. Fluxo de alteração

- 1 Alterar os ficheiros-fonte locais; não editar `public` diretamente.
- 2 Executar `npm run build` na raiz.
- 3 Testar as páginas afetadas.
- 4 Rever `git status` e `git diff` para confirmar exatamente o que mudou.
- 5 Fazer `commit` com mensagem clara.
- 6 Fazer `push` para main.
- 7 Verificar o deployment automático na Vercel.

4. Arquitetura atual de Atividades

Atividades é um módulo JavaScript integrado no portal central. O horário, sumários, questionários mensais e restantes registos partilhados residem no Supabase; APIs serverless executam operações que exigem validação adicional.



- A marcação da página é gerada por page.mjs e enriquecida pelo build central.
- app.js gere estado, Supabase no browser, diálogos, traduções, arrasto e geração de PDFs.
- As APIs validam o token Bearer e as permissões antes de usar a service role.
- RLS continua ativa nas tabelas acessíveis diretamente pelo cliente autenticado.
- public/ é artefacto de build; a fonte deve ser alterada em portal/, api/ e scripts/.

5. Mapa de ficheiros

Ficheiro	Responsabilidade
portal/modules/atividades/page.mjs	HTML dos controlos, calendário e diálogos de atividade, sumário, questionários, assinatura e indicadores.
portal/static/app.js	Estado e CRUD do horário, catálogo/monitores, sumários, questionários, indicadores, histórico, impressão e i18n.
portal/static/styles.css	Layout semanal, cartões, drag and drop, diálogos, impressão e temas.
api/activities-options.js	CRUD do catálogo/monitores e leitura/escrita do histórico com utilizador.
api/activities-summaries.js	Sumários, lista de utentes, presenças, assinaturas opcionais e sanitização.
api/activities-questionnaires.js	Referências, gravação, listagem, consulta e eliminação dos questionários mensais.
api/activities-statistics.js	Filtros semanal/mensal/anual, taxa de assiduidade, volume e horas de monitores.
portal/modules/atividades/supabase/schema.sql	Tabelas, índices, triggers, grants e políticas RLS.
supabase/20260721-activities-sensitive-permissions.sql	Migração da matriz atual: gestão do horário requer view_sensitive.
scripts/prepare-vercel-output.mjs	Monta menu, diálogos e páginas finais em public/ durante o build.
scripts/generate-manual-pdfs.py	Fonte editável dos dois manuais PDF de Atividades.

6. Modelo de dados Supabase

Tabela	Conteúdo	Observações
activities_catalog	Nomes selecionáveis das atividades e estado ativo.	Nome único; timestamps e created_by.
activities_monitors	Nome, telefone, email, NIF, voluntariado, profissão e descrição.	Detalhes privados; horas são calculadas, não guardadas num contador.
activities_schedule	id text, semana, dia, horas, título, teacher e sort_order.	teacher contém até dois nomes unidos por ' / '.
activities_summaries	Resumo, data, duração e attendance JSONB.	FK activity_id text; único por atividade/data; apagar cartão elimina o sumário.
activities_history	Ação, atividade, horários, semana, created_by e data.	A API acrescenta actor_name a partir do perfil central.
utente_abas	Questionário mensal em conteúdo JSONB, associado ao utente.	Usa tab_key determinística; não cria uma tabela nova nem usa localStorage.

Não altere o tipo de activities_schedule.id sem migrar a FK activities_summaries.activity_id. A incompatibilidade text/uuid impede criar a constraint.

7. Fluxo de leitura e escrita

- saveActivities e saveActivitiesHistory são no-op: não são a persistência de produção.
 - As chaves localStorage antigas existem apenas para uma migração única para Supabase e para preferências de idioma/tema.
 - Uma falha remota deve gerar erro visível; nunca fingir que uma alteração partilhada ficou guardada.
- 1 O módulo confirma a sessão central, carrega a matriz de permissões e cria o cliente Supabase autenticado.
 - 2 activities_schedule é lida diretamente com o token do utilizador e filtrada pela semana.
 - 3 Criar, editar, apagar, copiar e reordenar escrevem no Supabase e dependem de RLS view_sensitive.
 - 4 Catálogo, monitores e histórico usam activities-options com Authorization: Bearer <access token>.
 - 5 Sumários e utentes usam activities-summaries; questionários usam activities-questionnaires; indicadores e horas usam activities-statistics.
 - 6 Depois de cada gravação, o estado local é atualizado e o histórico recebe a ação e o nome do utilizador.

8. Matriz de permissões e RLS

- private.current_app_permission(area, action) lê public.app_users e reconhece administradores.
- A função private é SECURITY DEFINER com search_path fixo e execução apenas para authenticated.
- As APIs voltam a validar a permissão no servidor; esconder um botão nunca é a única barreira.
- activities-options pode devolver nomes a contas com view, mas só inclui detalhes privados do monitor com view_sensitive.
- activities-summaries permite consulta sanitizada com view; edição e assinaturas exigem edit.

Permissão	Interface/API	Persistência
atividades.view	Horário, olho, histórico, indicadores, manuais e nomes necessários.	SELECT schedule/history; APIs devolvem consulta sanitizada.

Permissão	Interface/API	Persistência
atividades.edit	Sumário, presenças e assinatura opcional.	CRUD summaries; API inclui lista de utentes e assinaturas ao editar.
atividades.view_sensitiv e	Criar/editar/apagar/arrastar/copiar; catálogo, detalhes de monitores e questionários mensais.	INSERT/UPDATE/DELETE schedule/catalog/monitors e CRUD de questionários via API.
atividades.export	Impressão da semana, sumário e indicadores.	Regista ações de impressão no histórico.
edit_sensitive/delete	Sem controlo próprio no módulo atual.	Não devem substituir view_sensitive sem uma migração coordenada.

9. Contratos e validação das APIs

- readBody deve limitar payloads; respostas de erro usam mensagens destinadas ao cliente e não expõem chaves.
- A service role fica exclusivamente em variáveis protegidas da Vercel, nunca em page.mjs, app.js ou PDFs.
- Assinaturas aceitam apenas data:image/png;base64 válida, com limites individual e agregado.
- IDs e nomes de presenças são reconstruídos a partir da tabela de Utentes; o browser não pode inventar participantes.
- Histórico calcula o nome visível da conta no servidor e não confia num actor_name enviado pelo cliente.

Endpoint	Métodos	Validação essencial
/api/activities-options	GET/POST/PATCH/DELETE	kind catalog/monitors/history, token, permissão, limites de texto e nomes de monitor sem '/
/api/activities-summaries	GET/POST	atividade existente, utentes permitidos, resumo até 20 000 caracteres e attendance normalizado.
/api/activities-questionnaires	GET/POST/DELETE	view_sensitive, atividade/utente existentes, período válido, 19 respostas de 1 a 5 e combinação única.
/api/activities-statistics	GET	period week/month/year, limites de datas, atividade opcional e exposição de presenças apenas com edit.

10. Regras do horário e compatibilidade

- A semana começa à segunda-feira e inclui apenas segunda a sexta.
- Inícios: 09:00 a 16:30; fins: 09:30 a 17:00; ambos em intervalos de 30 minutos.
- end_time tem de ser posterior a start_time, validado no cliente e por constraint SQL.
- Até dois monitores diferentes são guardados no campo teacher com separador ' / '. Preserve o parser ao renomear monitores.
- O Almoço é uma atividade normal criada por defeito em cada dia, das 12:00 às 13:00, inicialmente com Monitor a definir.
- Copiar semana anterior evita duplicados equivalentes e cria novos IDs/datas na semana de destino.
- sort_order controla a ordem de cartões; o drag and drop grava imediatamente a sequência.

Ao alterar nomes padrão, horários ou separadores, teste semanas antigas e a renomeação de monitores para não quebrar dados já guardados.

11. Sumários, presenças e assinaturas

- A janela recebe um `activity_id` fixo do cartão e apresenta metadados derivados do horário; não permite escolher outra atividade.
- `attendance` é um array JSONB com utente `id/name` e, opcionalmente, `signature/signatureAt`.
- Guardar presença não exige assinatura. A UI deve manter checkbox e canvas independentes.
- Ao desmarcar uma presença, remover também a assinatura associada no estado antes de gravar.
- A lista de Utentes é carregada no servidor apenas para contas com `edit`; uma conta só com `view` recebe o sumário sanitizado para o olho.
- Depois de guardar um sumário, o diálogo fecha e a consulta/indicador deve refletir os novos dados.
- A impressão usa dados persistidos; não deve depender de campos não guardados no DOM.

12. Questionários mensais

O questionário mensal reutiliza a infraestrutura existente de Utentes. Não existe uma tabela `activities_questionnaires` e não deve ser introduzida persistência em `localStorage`.

- A UI só monta as perguntas depois de receber atividade, utente, mês e ano válidos.
- Antes de gravar, a API normaliza o período e rejeita respostas em falta, fora de 1-5 ou contextos inexistentes.
- A combinação já guardada é detetada antes do preenchimento e encaminha para Ver questionário realizado, evitando duplicados.
- O separador Média filtra exatamente por mês e ano e, opcionalmente, por atividade; Todas as atividades cria um grupo por atividade.
- A média de cada secção usa apenas as chaves dessa secção, ignora valores inválidos e é formatada com uma casa decimal na escala de 1 a 5.
- A consulta individual reutiliza o mesmo cálculo para apresentar a média no cabeçalho de cada secção, sem gravar campos derivados.
- A eliminação exige confirmação na interface e autorização novamente no servidor.
- `createdBy` e `updatedBy` são derivados da sessão autenticada; não confiar em identidade enviada pelo browser.
- As traduções PT/EN devem cobrir separadores, filtros, 19 perguntas, escala, estados vazios, consulta e confirmações.

Elemento	Contrato técnico
Referências	Atividades ativas vêm de <code>activities_catalog</code> ; utentes vêm de <code>utentes</code> . A API valida novamente IDs e nomes no servidor.
Identificador	<code>tab_key = activities_questionnaire:<activityId>:<AAAA-MM></code> , único dentro do utente em <code>public.utente_abas</code> .
Conteúdo	conteudo JSONB com kind <code>activity_questionnaire</code> , version 1, atividade, utente, mês/ano, responses e metadados de criação/atualização.
Respostas	19 chaves estáveis, todas obrigatórias, com inteiro de 1 a 5. Não renomear chaves ao ajustar o texto visível.
Médias	A interface calcula no cliente a média geral e a média de cada secção a partir de responses já persistido, sem criar outra tabela.
Autorização	GET, POST e DELETE exigem sessão central verificada e <code>atividades.view_sensitive</code> ; a <code>service role</code> fica apenas no servidor.
Consulta	A lista aceita filtros; Abrir usa um estado separado e somente de leitura. O registo concluído não regressa ao formulário inicial.

13. Indicadores e horas de monitores

- Semanal usa `weekStart`; mensal usa mês/ano; anual esconde o mês e usa apenas o ano.
- O filtro de atividade compara o título guardado; renomes de catálogo não reescrevem automaticamente históricos antigos.
- Presenças e taxa de assiduidade só são expostas a quem tem edit; utilizadores só com view recebem totais sem a lista pessoal.
- As horas não são um contador mutável: são recalculadas, evitando divergências ao editar ou apagar atividades.

Indicador	Cálculo
Atividades	Número de linhas schedule dentro do intervalo e filtro escolhidos.
Sumários	Número de summaries no período.
Média de utentes	Total de presenças dividido pelo número de sumários.
Taxa de assiduidade	Tempo efetivamente presente / duração total das atividades em que o utente marcou presença x 100.
Volume	Soma de duração da sessão x número de presentes, em minutos convertidos para horas-pessoa.
Horas do monitor	Soma da duração das atividades schedule em que o nome aparece como primeiro ou segundo monitor.

14. Histórico, tradução e impressão

- O histórico guarda created, updated, deleted, reordered, printed, summary_saved e summary_printed conforme a ação.
- A página mostra Data, Utilizador, Ação, Atividade e Detalhes; o modo escuro deve preservar contraste.
- Todas as strings visíveis dos diálogos e cartões precisam de chaves PT/EN em portal/static/app.js.
- Novos títulos de página devem seguir 'Atividades | MenteMovimento' e a tradução ativa.
- A impressão semanal e dos sumários usa pdfMake com fallback de download para tablets; não chamar window.print sobre a página completa.
- A impressão de indicadores deve reproduzir os filtros e valores apresentados no diálogo.

15. Alterar código manualmente

- 1 Confirmar o estado do trabalho e criar uma branch/commit identificável antes de alterações grandes.
- 2 Localizar marcação, tradução, listener, API, schema e CSS relacionados com rg; uma funcionalidade atravessa vários ficheiros.
- 3 Editar apenas as fontes em portal/, api/, api-lib/, supabase/ e scripts/; não tratar public/ como fonte.
- 4 Se o contrato de dados mudar, escrever SQL compatível com tabelas existentes e preservar tipos/FKs.
- 5 Atualizar PT/EN, permissões, histórico, impressão e estes manuais na mesma alteração.
- 6 Rever git diff e executar testes antes de gerar novamente public/.

```
git status --short
rg -n "activities\.[data-activities|activities_" portal api scripts supabase
git diff --check
npm run build
```

16. Testes mínimos por alteração

Área	Testes obrigatórios
Horário	Semanas anterior/seguinte, datas, criar, editar, apagar, copiar, almoço, dois monitores e ordem por arrasto.
Permissões	Contas só view, view+edit, view_sensitive e export; confirmar botões e bloqueio real de API/RLS.
Sumário	Criar/editar, presenças sem assinatura, assinatura tablet, desmarcar, fechar ao guardar e olho em consulta.
Questionários	Quatro seleções obrigatórias, 19 respostas, duplicado, filtros, consulta só de leitura, eliminação confirmada, permissões, PT/EN e tablet.
Catálogo/monitores	CRUD, renomear monitor usado, dados privados, mês/ano atual e horas calculadas.
Indicadores	Semana/mês/ano, todas/uma atividade, taxa de assiduidade, volume, zero dados e impressão.
Histórico	Nome do utilizador, todas as ações, tradução e tema escuro.
Impressão	Desktop e tablet; semana numa folha, sumário/participantes, indicadores e fallback PDF.
UI	Desktop/tablet, claro/escuro, PT/EN, menus exclusivos e clique fora para fechar.

17. Aplicar SQL no Supabase

- Os questionários não exigem uma tabela nova: dependem de public.utente_abas já existente e da respetiva RLS/API.
 - Não usar DROP ... CASCADE para resolver dependências sem um plano de migração.
 - Não mudar nomes de parâmetros de funções existentes com CREATE OR REPLACE; remova/recrie de forma controlada ou preserve a assinatura.
 - Políticas dependem de private.current_app_permission(text, text); alterar essa função exige considerar todas as policies dependentes.
 - Guardar a migração no repositório depois de validada; nunca depender apenas do histórico do SQL Editor.
- 1 Fazer backup e confirmar que o SQL Editor está aberto no projeto Supabase da produção correta.
 - 2 Em instalação nova, executar portal/modules/atividades/supabase/schema.sql por completo.
 - 3 Numa base existente, rever e executar a migração datada necessária, por exemplo 20260721-activities-sensitive-permissions.sql.

- 4 Escolher Run and enable RLS quando o editor alertar para tabelas sem RLS; o schema também ativa RLS explicitamente.
- 5 Confirmar Success, executar notify pgrst, 'reload schema' quando necessário e voltar a testar com contas de permissões diferentes.

18. Gerar e verificar os manuais

Os PDFs são artefactos gerados. A fonte editável é `scripts/generate-manual-pdfs.py`.

- Confirmar os dois PDFs em `portal/modules/atividades/docs/`.
- Renderizar todas as páginas em PNG e rever cortes, sobreposições, acentos, tabelas e código.
- Procurar descrições antigas como professor, `localStorage` como armazenamento principal ou falta de Supabase.
- Executar `npm run build` para copiar os PDFs para `public/area/atividades/docs/`.

```
python scripts/generate-manual-pdfs.py --only atividades
pdfinfo portal/modules/atividades/docs/Manual_Utilizador_Atividades.pdf
pdftotext portal/modules/atividades/docs/Manual_Programador_Atividades.pdf -
```

19. Build, Git e publicação

- 1 Executar `npm install` apenas quando as dependências mudarem e confirmar o lockfile.
- 2 Executar `npm run build` na raiz e verificar `public/area/atividades/` e os PDFs gerados.
- 3 Testar localmente login, horário, API, permissões, PDFs e fluxos principais.
- 4 Rever `git status`, `git diff --check` e o diff dos ficheiros alterados; remover segredos e dados reais.
- 5 Criar commit claro e enviar para main conforme o processo da equipa.
- 6 Aguardar o deployment automático ou executar `npx vercel deploy --prod --yes` numa sessão autorizada.
- 7 Na produção, repetir testes rápidos e confirmar o deployment Ready antes de encerrar.

```
npm run build
git status --short
git diff --check
git add <ficheiros-revistos>
git commit -m "Atualizar modulo de atividades"
git push origin main
```

20. Segurança, diagnóstico e rollback

- Para rollback de código, promover o deployment anterior ou reverter o commit sem apagar trabalho alheio.
- Para dados, preferir migração corretiva testada com backup; não repor toda a base por causa de um erro isolado.
- Antes de fechar: build verde, RLS/API testadas, PDFs revistos, produção Ready e nenhum token no repositório.

Sintoma	Verificação
Tabela/campo não encontrado	Aplicar schema/migração no projeto correto e recarregar o cache PostgREST.
RLS bloqueia	Confirmar <code>app_users</code> , action atual, função <code>private</code> e <code>JWT auth.uid()</code> .
Atividade não partilha	Verificar Supabase URL/anon key, sessão, RLS e erro de <code>activities_schedule</code> ; não reativar <code>localStorage</code> .
API 401/403	Confirmar Authorization Bearer, validade da sessão e permissão exigida.
API 500	Confirmar service role na Vercel, logs sem dados pessoais, tabela/colunas e limites do payload.
Impressão tablet errada	Confirmar <code>pdfMake</code> carregado, fallback de download e que não é usado <code>window.print</code> da página.
Horas/indicadores erradas	Rever datas, nomes de monitor, duração, sumários, presenças e filtro selecionado.
Questionários não carregam	Confirmar <code>utente_abas</code> , referências ativas, sessão verificada, <code>view_sensitive</code> , resposta da API e cache PostgREST; não criar tabela paralela.